

lab_music_partial

November 12, 2025

1 Lab: Neural Networks for Music Classification

In addition to the concepts in the [MNIST neural network demo](#), in this lab, you will learn to: *

- Load a file from a URL
- * Extract simple features from audio samples for machine learning tasks such as speech recognition and classification
- * Build a simple neural network for music classification using these features
- * Use a callback to store the loss and accuracy history in the training process
- * Optimize the learning rate of the neural network

To illustrate the basic concepts, we will look at a relatively simple music classification problem. Given a sample of music, we want to determine which instrument (e.g. trumpet, violin, piano) is playing. This dataset was generously supplied by [Prof. Juan Bello](#) at NYU Stenhardt and his former PhD student Eric Humphrey (now at Spotify). They have a complete website dedicated to deep learning methods in music informatics:

<http://marl.smusic.nyu.edu/wordpress/projects/feature-learning-deep-architectures/deep-learning-python-tutorial/>

You can also check out Juan's course.

We first load the standard packages

```
[1]: import numpy as np
import matplotlib
import matplotlib.pyplot as plt
```

1.1 Audio Feature Extraction with Librosa

The key to audio classification is to extract the correct features. In addition to `pytorch`, we will need the `librosa` package. The `librosa` package in python has a rich set of methods extracting the features of audio samples commonly used in machine learning tasks such as speech recognition and sound classification.

Installation instructions and complete documentation for the package are given on the [librosa main page](#). On most systems, you should be able to simply use:

```
pip install -u librosa
```

If you are using Google colab, the packages are already installed. After you have installed the package, try to import it.

```
[9]: import librosa
import librosa.display
import librosa.feature
```

In this lab, we will use a set of music samples from the website:

<http://theremin.music.uiowa.edu>

This website has a great set of samples for audio processing. Look on the web for how to use the `requests.get` and `file.write` commands to load the file at the URL provided into your working directory.

You can play the audio sample by copying the file to your local machine and playing it on any media player. If you listen to it you will hear a soprano saxophone (with vibrato) playing four notes (C, C#, D, Eb).

```
[11]: import requests
fn = "SopSax.Vib.pp.C6Eb6.aiff"
url = "http://theremin.music.uiowa.edu/sound files/MIS/Woodwinds/
      ↪sopranosaxophone/"+fn

#Load the file from url and save it in a file under the name fn
loaded_file = requests.get(url)
with open(fn, 'wb') as f:
    f.write(loaded_file.content)

print(loaded_file)
```

<Response [200]>

Next, use `librosa` command `librosa.load` to read the audio file with filename `fn` and get the samples `y` and sample rate `sr`.

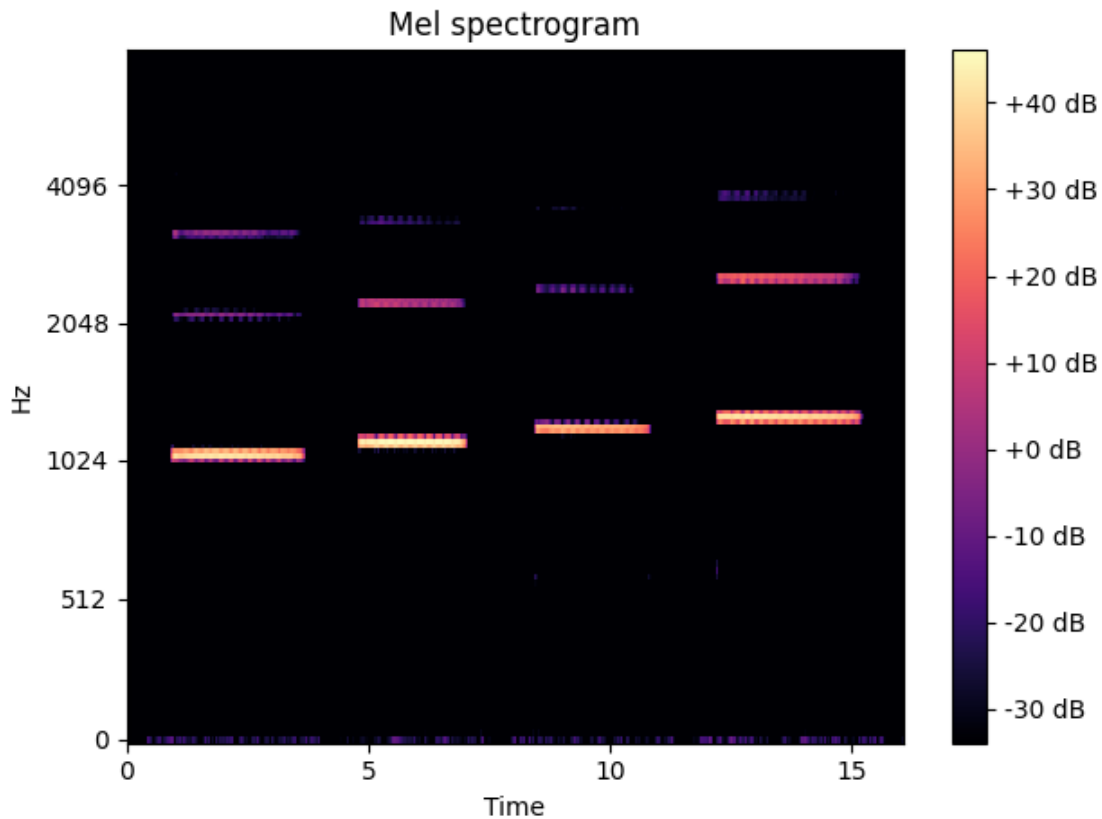
```
[12]: # TODO
y, sr = librosa.load(fn)
print(y)
print(sr)
```

```
[ 0.          0.          0.          ... -0.00370976 -0.00431869
 -0.00290983]
22050
```

Extracting features from audio files is an entire subject on its own right. A commonly used set of features are called the Mel Frequency Cepstral Coefficients (MFCCs). These are derived from the so-called mel spectrogram which is something like a regular spectrogram, but the power and frequency are represented in log scale, which more naturally aligns with human perceptual processing. You can run the code below to display the mel spectrogram from the audio sample.

You can easily see the four notes played in the audio track. You also see the ‘harmonics’ of each notes, which are other tones at integer multiples of the fundamental frequency of each note.

```
[13]: S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128, fmax=8000)
librosa.display.specshow(librosa.amplitude_to_db(S),
                        y_axis='mel', fmax=8000, x_axis='time')
plt.colorbar(format='%+2.0f dB')
plt.title('Mel spectrogram')
plt.tight_layout()
```



1.2 Downloading the Data

Using the MFCC features described above, Eric Humphrey and Juan Bellow have created a complete data set that can be used for instrument classification. Essentially, they collected a number of data files from the website above. For each audio file, they then segmented the track into notes and then extracted 120 MFCCs for each note. The goal is to recognize the instrument from the 120 MFCCs. The process of feature extraction is quite involved. So, we will just use their processed data provided at:

<https://github.com/marl/dl4mir-tutorial/blob/master/README.md>

Note the password. Load the four files into some directory, say `instrument_dataset`. Then, load them with the commands.

```
[16]: data_dir = 'instrument_dataset/'
import os
fn = os.path.join(data_dir, 'uiowa_train_data.npy')
Xtr = np.load(fn)
fn = os.path.join(data_dir, 'uiowa_train_labels.npy')
ytr = np.load(fn)
fn = os.path.join(data_dir, 'uiowa_test_data.npy')
Xts = np.load(fn)
fn = os.path.join(data_dir, 'uiowa_test_labels.npy')
yts = np.load(fn)
```

Looking at the data files: * What are the number of training and test samples? * What is the number of features for each sample? * How many classes (i.e. instruments) are there per class?

```
[21]: # TODO using the data files
ntr = Xtr.shape[0]
nts = Xts.shape[0]
nfeatures = Xtr.shape[1]
nclasses = len(np.unique(ytr))

print(f"Number of training samples: {ntr}")
print(f"Number of test samples: {nts}")
print(f"Number of features per sample: {nfeatures}")
print(f"Number of classes: {nclasses}")
```

```
Number of training samples: 66247
Number of test samples: 14904
Number of features per sample: 120
Number of classes: 10
```

Before continuing, you must scale the training and test data, `Xtr` and `Xts`. Compute the mean and std deviation of each feature in `Xtr` and create a new training data set, `Xtr_scale`, by subtracting the mean and dividing by the std deviation. Also compute a scaled test data set, `Xts_scale` using the mean and std deviation learned from the training data set.

```
[22]: # TODO Scale the training and test matrices
mean = np.mean(Xtr, axis=0)
std = np.std(Xtr, axis=0)
Xtr_scale = (Xtr - mean) / std
Xts_scale = (Xts - mean) / std
```

1.3 Building a Neural Network Classifier

Following the example in [MNIST neural network demo](#). create a neural network model with: * `nfeatures` inputs * `nh=256` hidden units * `sigmoid` activation * fully connected layer with `nclasses` outputs * `nn.Softmax` output * select the input and output shapes correctly

```
[23]: import torch
import torch.nn as nn
```

```

import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self, nfeatures, nh, nclasses):
        super(Model, self).__init__()
        self.fc1 = nn.Linear(nfeatures, nh)
        self.fc2 = nn.Linear(nh, nclasses)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

nh = 256
model = Model(nfeatures, nh, nclasses)

```

Now print a summary of the model. You can use the `torchsummary.summary` method as follows:

```

[24]: from torchsummary import summary

# TODO
summary(model, (nfeatures,))

```

```

-----
              Layer (type)              Output Shape          Param #
=====
              Linear-1                  [-1, 256]              30,976
              Linear-2                  [-1, 10]               2,570
=====
Total params: 33,546
Trainable params: 33,546
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.13
Estimated Total Size (MB): 0.13
-----

```

Next, create `pytorch Dataloader` classes from the `numpy` arrays. Note that you must use the scaled data `Xtr_scale` and `Xts_scale`.

```

[25]: from torch.utils.data import DataLoader, TensorDataset

Xtr_torch = torch.from_numpy(Xtr_scale).float()
Xts_torch = torch.from_numpy(Xts_scale).float()
ytr_torch = torch.from_numpy(ytr).long()
yts_torch = torch.from_numpy(yts).long()

train_dataset = TensorDataset(Xtr_torch, ytr_torch)

```

```
test_dataset = TensorDataset(Xts_torch, yts_torch)

batch_size = 100
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

Create the loss and an optimizer. For the loss, use `nn.CrossEntropyLoss()`. For the optimizer, use the `optim.Adam` optimizer with a learning rate of 0.001

```
[26]: import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Fit the model for 20 epochs using the scaled data for both the training and validation. In each epoch, save the: * average training loss * training accuracy * test accuracy

Your final test accuracy should be >95%.

```
[28]: # Define a function to compute accuracy
def accuracy(output, target):
    # Get predicted class by taking the class with the highest logit
    preds = output.argmax(dim=1)
    correct = (preds == target).float().sum()
    return correct / len(target)

# Lists to store training and testing accuracy for each epoch

train_accuracy_history = []
test_accuracy_history = []
loss_history = []

# Training loop
epochs = 20

for epoch in range(epochs):
    model.train()
    correct = 0
    total = 0
    running_loss = 0.0

    for (data, target) in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * data.size(0)
        preds = output.argmax(dim=1)
```

```

        correct += (preds == target).sum().item()
        total += target.size(0)

    avg_loss = running_loss / total
    train_acc = correct / total
    loss_history.append(avg_loss)
    train_accuracy_history.append(train_acc)

    # Evaluate on test set
    model.eval()
    correct_test = 0
    total_test = 0
    with torch.no_grad():
        for (data, target) in test_loader:
            output = model(data)
            preds = output.argmax(dim=1)
            correct_test += (preds == target).sum().item()
            total_test += target.size(0)
    test_acc = correct_test / total_test
    test_accuracy_history.append(test_acc)

    # Print accuracy and loss for this epoch
    print(f"Epoch {epoch+1}/{epochs}, Training Loss: {avg_loss:.4f}, Training_
↳Accuracy: {train_acc:.4f}, Test Accuracy: {test_acc:.4f}")

```

```

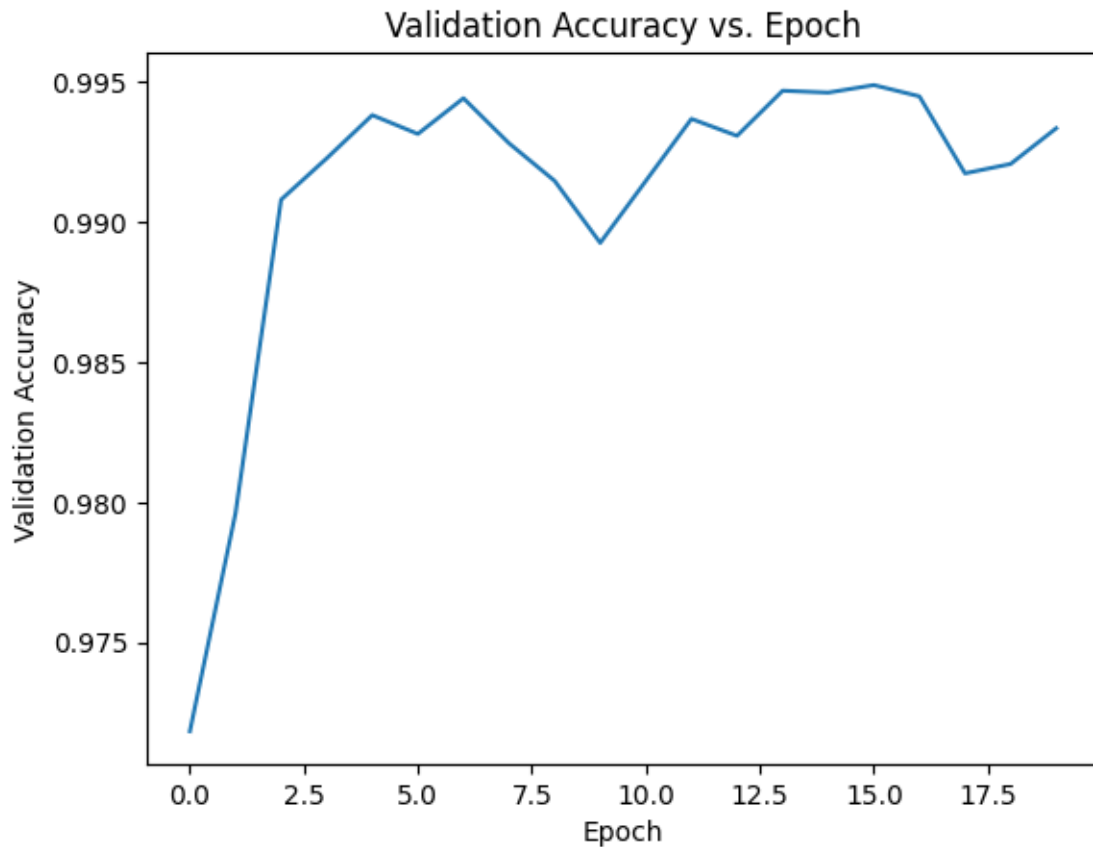
Epoch 1/20, Training Loss: 0.1958, Training Accuracy: 0.9433, Test Accuracy:
0.9718
Epoch 2/20, Training Loss: 0.0423, Training Accuracy: 0.9884, Test Accuracy:
0.9796
Epoch 3/20, Training Loss: 0.0269, Training Accuracy: 0.9920, Test Accuracy:
0.9908
Epoch 4/20, Training Loss: 0.0182, Training Accuracy: 0.9947, Test Accuracy:
0.9923
Epoch 5/20, Training Loss: 0.0153, Training Accuracy: 0.9955, Test Accuracy:
0.9938
Epoch 6/20, Training Loss: 0.0118, Training Accuracy: 0.9965, Test Accuracy:
0.9932
Epoch 7/20, Training Loss: 0.0107, Training Accuracy: 0.9966, Test Accuracy:
0.9944
Epoch 8/20, Training Loss: 0.0085, Training Accuracy: 0.9974, Test Accuracy:
0.9928
Epoch 9/20, Training Loss: 0.0090, Training Accuracy: 0.9973, Test Accuracy:
0.9915
Epoch 10/20, Training Loss: 0.0060, Training Accuracy: 0.9983, Test Accuracy:
0.9893
Epoch 11/20, Training Loss: 0.0068, Training Accuracy: 0.9978, Test Accuracy:
0.9915
Epoch 12/20, Training Loss: 0.0066, Training Accuracy: 0.9982, Test Accuracy:

```

0.9937
Epoch 13/20, Training Loss: 0.0063, Training Accuracy: 0.9983, Test Accuracy: 0.9931
Epoch 14/20, Training Loss: 0.0047, Training Accuracy: 0.9985, Test Accuracy: 0.9947
Epoch 15/20, Training Loss: 0.0049, Training Accuracy: 0.9985, Test Accuracy: 0.9946
Epoch 16/20, Training Loss: 0.0045, Training Accuracy: 0.9987, Test Accuracy: 0.9949
Epoch 17/20, Training Loss: 0.0037, Training Accuracy: 0.9989, Test Accuracy: 0.9945
Epoch 18/20, Training Loss: 0.0052, Training Accuracy: 0.9983, Test Accuracy: 0.9917
Epoch 19/20, Training Loss: 0.0032, Training Accuracy: 0.9992, Test Accuracy: 0.9921
Epoch 20/20, Training Loss: 0.0032, Training Accuracy: 0.9991, Test Accuracy: 0.9934

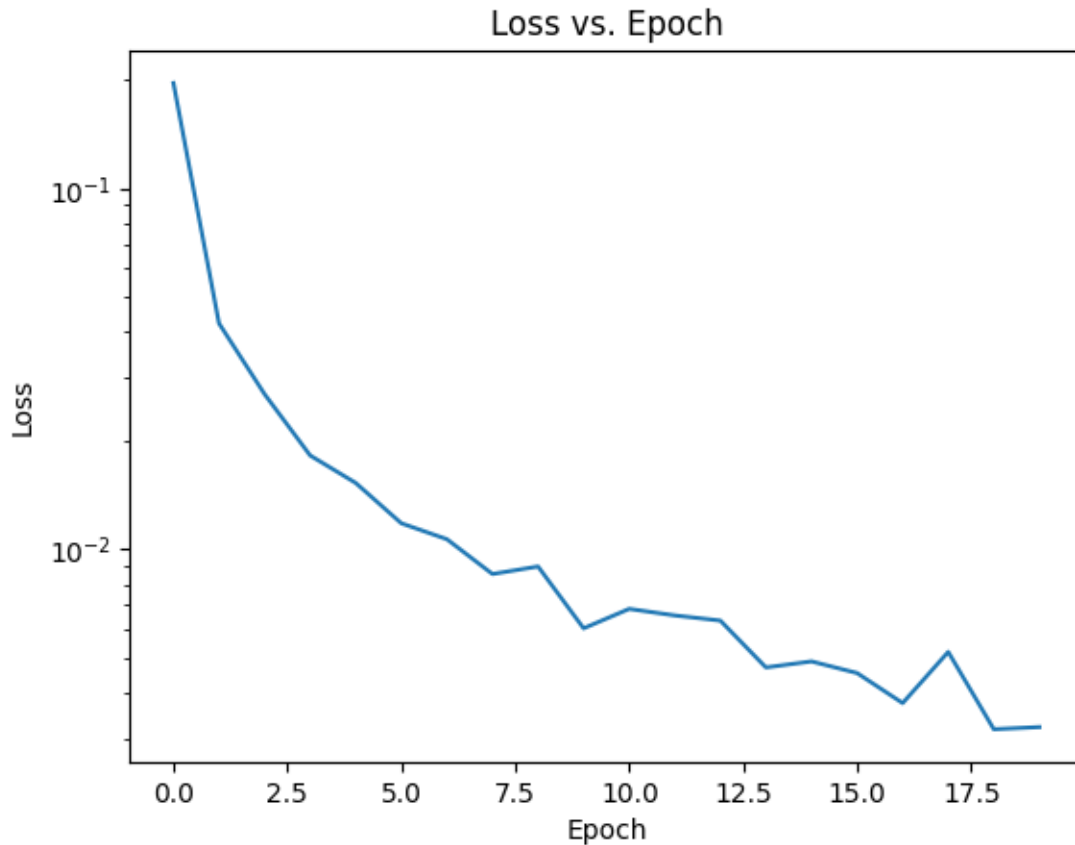
Plot the validation accuracy saved in `test_accuracy_history`.

```
[30]: plt.plot(test_accuracy_history)
      plt.xlabel('Epoch')
      plt.ylabel('Validation Accuracy')
      plt.title('Validation Accuracy vs. Epoch')
      plt.show()
```



Plot the loss values saved in the `loss_history`. You should see that the loss is steadily decreasing. Use the `semilogy` plot.

```
[32]: # TODO
plt.semilogy(loss_history)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss vs. Epoch')
plt.show()
```



1.4 Optimizing the Learning Rate

One challenge in training neural networks is the selection of the learning rate. Rerun the above code, trying four learning rates as shown in the vector `rates`. For each learning rate: * construct the network * select the optimizer. Use the Adam optimizer with the appropriate learning rate. * train the model for 20 epochs * Save the accuracy and losses

```
[38]: rates = [1e-4, 1e-3, 1e-2]
test_acc_lr = []
loss_hist_lr = []

# TODO
for lr in rates:

    # TODO: Create the model
    # model = Model(...)
    model = Model(nfeatures, nh, nclasses)

    # TODO: Define loss function and optimizer
```

```

# criterion = ...
# optimizer = ...
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

# Lists to store training and testing accuracy for each epoch
train_accuracy_history = []
test_accuracy_history = []
loss_history = []

# Number of epochs
epochs = 20

# Training loop
for epoch in range(epochs):

    # TODO: Train over epoch
    model.train()
    running_loss = 0.0
    train_correct = 0
    train_total = 0

    for (data,target) in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        # Accumulate loss and accuracy
        running_loss += loss.item() * data.size(0)
        preds = output.argmax(dim=1)
        train_correct += (preds == target).sum().item()
        train_total += target.size(0)

    # Compute average loss and accuracy for this epoch
    avg_loss = running_loss / train_total
    train_acc = train_correct / train_total
    loss_history.append(avg_loss)
    train_accuracy_history.append(train_acc)

    # TODO: Test over epoch
    model.eval()
    test_correct = 0
    test_total = 0
    with torch.no_grad():

```

```

        for (data,target) in test_loader:
            output = model(data)
            preds = output.argmax(dim=1)
            test_correct += (preds == target).sum().item()
            test_total += target.size(0)

    test_acc = test_correct / test_total
    test_accuracy_history.append(test_acc)

    # TODO: Print accuracy
    print(f"Epoch {epoch+1}/{epochs}, Training Loss: {avg_loss:.4f}, Training_
Accuracy: {train_acc:.4f}, Test Accuracy: {test_acc:.4f}")

    # TODO: Save the loss history (after all epochs for this learning rate)
    loss_hist_lr.append(loss_history)
    test_acc_lr.append(test_accuracy_history)

```

```

Epoch 1/20, Training Loss: 0.8587, Training Accuracy: 0.7449, Test Accuracy:
0.8289
Epoch 2/20, Training Loss: 0.2820, Training Accuracy: 0.9326, Test Accuracy:
0.9153
Epoch 3/20, Training Loss: 0.1710, Training Accuracy: 0.9568, Test Accuracy:
0.9403
Epoch 4/20, Training Loss: 0.1223, Training Accuracy: 0.9692, Test Accuracy:
0.9557
Epoch 5/20, Training Loss: 0.0948, Training Accuracy: 0.9762, Test Accuracy:
0.9653
Epoch 6/20, Training Loss: 0.0769, Training Accuracy: 0.9808, Test Accuracy:
0.9717
Epoch 7/20, Training Loss: 0.0647, Training Accuracy: 0.9839, Test Accuracy:
0.9805
Epoch 8/20, Training Loss: 0.0555, Training Accuracy: 0.9860, Test Accuracy:
0.9855
Epoch 9/20, Training Loss: 0.0484, Training Accuracy: 0.9876, Test Accuracy:
0.9868
Epoch 10/20, Training Loss: 0.0428, Training Accuracy: 0.9893, Test Accuracy:
0.9876
Epoch 11/20, Training Loss: 0.0382, Training Accuracy: 0.9903, Test Accuracy:
0.9880
Epoch 12/20, Training Loss: 0.0346, Training Accuracy: 0.9913, Test Accuracy:
0.9890
Epoch 13/20, Training Loss: 0.0314, Training Accuracy: 0.9920, Test Accuracy:
0.9899
Epoch 14/20, Training Loss: 0.0286, Training Accuracy: 0.9931, Test Accuracy:
0.9905
Epoch 15/20, Training Loss: 0.0262, Training Accuracy: 0.9934, Test Accuracy:

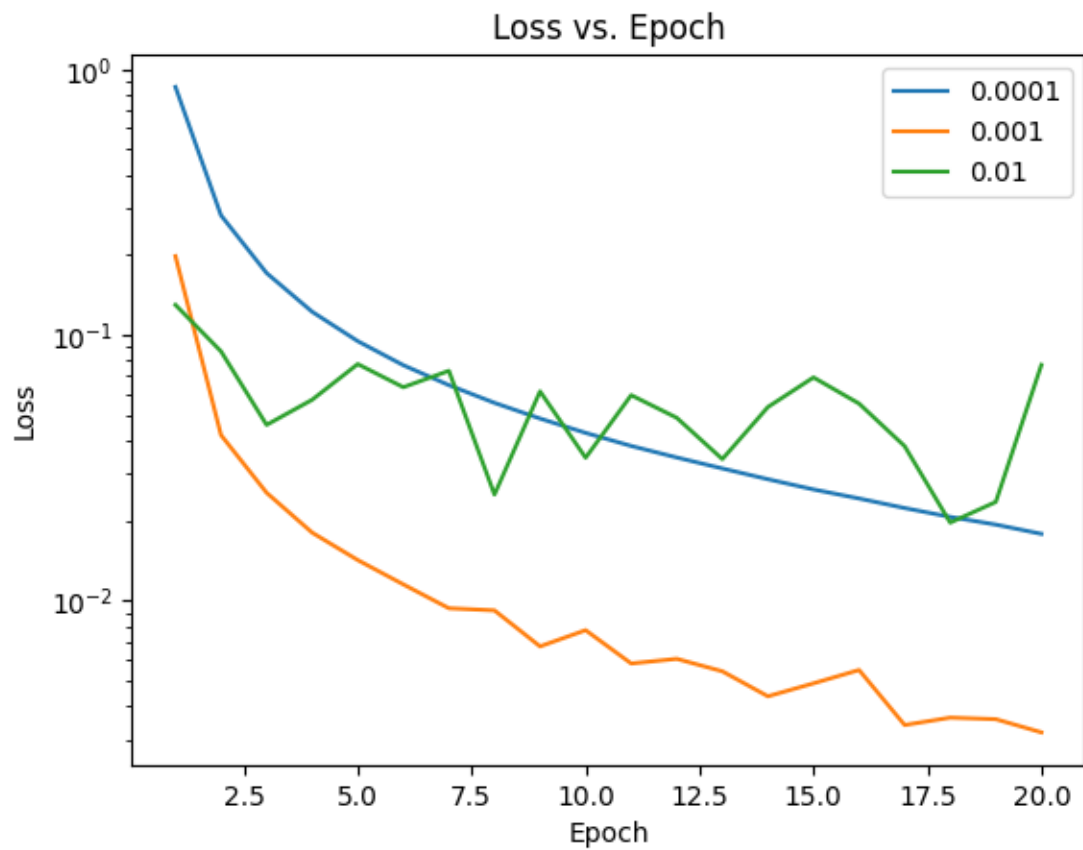
```

0.9901
 Epoch 16/20, Training Loss: 0.0242, Training Accuracy: 0.9938, Test Accuracy: 0.9911
 Epoch 17/20, Training Loss: 0.0223, Training Accuracy: 0.9941, Test Accuracy: 0.9897
 Epoch 18/20, Training Loss: 0.0206, Training Accuracy: 0.9948, Test Accuracy: 0.9888
 Epoch 19/20, Training Loss: 0.0193, Training Accuracy: 0.9952, Test Accuracy: 0.9901
 Epoch 20/20, Training Loss: 0.0178, Training Accuracy: 0.9953, Test Accuracy: 0.9907
 Epoch 1/20, Training Loss: 0.1980, Training Accuracy: 0.9426, Test Accuracy: 0.9790
 Epoch 2/20, Training Loss: 0.0420, Training Accuracy: 0.9887, Test Accuracy: 0.9886
 Epoch 3/20, Training Loss: 0.0255, Training Accuracy: 0.9929, Test Accuracy: 0.9803
 Epoch 4/20, Training Loss: 0.0180, Training Accuracy: 0.9948, Test Accuracy: 0.9890
 Epoch 5/20, Training Loss: 0.0142, Training Accuracy: 0.9956, Test Accuracy: 0.9883
 Epoch 6/20, Training Loss: 0.0115, Training Accuracy: 0.9964, Test Accuracy: 0.9912
 Epoch 7/20, Training Loss: 0.0093, Training Accuracy: 0.9972, Test Accuracy: 0.9905
 Epoch 8/20, Training Loss: 0.0092, Training Accuracy: 0.9972, Test Accuracy: 0.9915
 Epoch 9/20, Training Loss: 0.0067, Training Accuracy: 0.9981, Test Accuracy: 0.9924
 Epoch 10/20, Training Loss: 0.0077, Training Accuracy: 0.9978, Test Accuracy: 0.9924
 Epoch 11/20, Training Loss: 0.0058, Training Accuracy: 0.9981, Test Accuracy: 0.9879
 Epoch 12/20, Training Loss: 0.0060, Training Accuracy: 0.9981, Test Accuracy: 0.9927
 Epoch 13/20, Training Loss: 0.0054, Training Accuracy: 0.9984, Test Accuracy: 0.9921
 Epoch 14/20, Training Loss: 0.0044, Training Accuracy: 0.9987, Test Accuracy: 0.9910
 Epoch 15/20, Training Loss: 0.0049, Training Accuracy: 0.9984, Test Accuracy: 0.9904
 Epoch 16/20, Training Loss: 0.0055, Training Accuracy: 0.9984, Test Accuracy: 0.9897
 Epoch 17/20, Training Loss: 0.0034, Training Accuracy: 0.9990, Test Accuracy: 0.9913
 Epoch 18/20, Training Loss: 0.0036, Training Accuracy: 0.9988, Test Accuracy: 0.9906
 Epoch 19/20, Training Loss: 0.0036, Training Accuracy: 0.9989, Test Accuracy:

0.9918
Epoch 20/20, Training Loss: 0.0032, Training Accuracy: 0.9991, Test Accuracy: 0.9925
Epoch 1/20, Training Loss: 0.1298, Training Accuracy: 0.9639, Test Accuracy: 0.9764
Epoch 2/20, Training Loss: 0.0867, Training Accuracy: 0.9819, Test Accuracy: 0.9913
Epoch 3/20, Training Loss: 0.0458, Training Accuracy: 0.9904, Test Accuracy: 0.9854
Epoch 4/20, Training Loss: 0.0570, Training Accuracy: 0.9897, Test Accuracy: 0.9747
Epoch 5/20, Training Loss: 0.0777, Training Accuracy: 0.9888, Test Accuracy: 0.9767
Epoch 6/20, Training Loss: 0.0636, Training Accuracy: 0.9916, Test Accuracy: 0.9439
Epoch 7/20, Training Loss: 0.0733, Training Accuracy: 0.9913, Test Accuracy: 0.9853
Epoch 8/20, Training Loss: 0.0251, Training Accuracy: 0.9954, Test Accuracy: 0.9785
Epoch 9/20, Training Loss: 0.0613, Training Accuracy: 0.9922, Test Accuracy: 0.9773
Epoch 10/20, Training Loss: 0.0345, Training Accuracy: 0.9950, Test Accuracy: 0.9850
Epoch 11/20, Training Loss: 0.0593, Training Accuracy: 0.9937, Test Accuracy: 0.9829
Epoch 12/20, Training Loss: 0.0487, Training Accuracy: 0.9944, Test Accuracy: 0.9820
Epoch 13/20, Training Loss: 0.0341, Training Accuracy: 0.9962, Test Accuracy: 0.9855
Epoch 14/20, Training Loss: 0.0535, Training Accuracy: 0.9947, Test Accuracy: 0.9808
Epoch 15/20, Training Loss: 0.0692, Training Accuracy: 0.9942, Test Accuracy: 0.9826
Epoch 16/20, Training Loss: 0.0551, Training Accuracy: 0.9953, Test Accuracy: 0.9851
Epoch 17/20, Training Loss: 0.0381, Training Accuracy: 0.9954, Test Accuracy: 0.9899
Epoch 18/20, Training Loss: 0.0196, Training Accuracy: 0.9971, Test Accuracy: 0.9917
Epoch 19/20, Training Loss: 0.0235, Training Accuracy: 0.9968, Test Accuracy: 0.9920
Epoch 20/20, Training Loss: 0.0771, Training Accuracy: 0.9948, Test Accuracy: 0.9875

Plot the loss function vs. the epoch number for all three learning rates on one graph. You should see that the lower learning rates are more stable, but converge slower. The higher learning rate either plateaus at a lower validation accuracy or is unstable.

```
[39]: # TODO
epochs = range(1, len(loss_hist_lr[0]) + 1) # Epoch numbers: 1, 2, ..., 20
for i, lr in enumerate(rates):
    plt.semilogy(epochs, loss_hist_lr[i], label=str(lr))
plt.legend()
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss vs. Epoch')
plt.show()
```



[]:

[]: